POKHARA UNIVERSITY

GANDAKI COLLEGE OF ENGINEERING AND SCIENCE

LABORATORY REPORT

AGILE SOFTWARE DEVELOPMENT LAB

Lab No. 2

BESE III YEAR - VI SEM

Submitted By:

Sudip Sharma

Roll No: 39

Program: BESE ,2023

Submitted To:

Er. Rajendra Bahadur Thapa

Lecturer

Date: July 3, 2026

NAME OF EXPERIMENT

Implementation and Exploration of Different Features of an Integrated Development Environment (IDE) using Visual Studio Code.

AIM

To study and explore the important features of Visual Studio Code (VS Code), including Build (Debug and Release), Breakpoints, Refactoring, Source Code Collaboration using Git, Command Palette, and IntelliSense.

THEORY

An **Integrated Development Environment (IDE)** is a comprehensive software application that provides developers with all the essential tools required to write, test, debug, and manage software efficiently. Instead of using separate tools for editing, compiling, and debugging, an IDE combines them into a single, unified graphical interface. This integration significantly boosts developer productivity, reduces setup complexity, and helps in managing large codebases.

Visual Studio Code (VS Code) is a lightweight yet powerful open-source IDE developed by Microsoft. It is widely used in the software industry because of its cross-platform support, extensive extension marketplace, and built-in support for modern development workflows. VS Code supports multiple programming languages including Python, JavaScript, C++, and Java through various extensions.

The key features explored in this lab are:

1. **Build (Debug and Release)**: Building is the process of converting human-readable source code into an executable program that the computer can run.
    - *Debug Build*: This configuration is used during the active development phase. It includes additional debugging symbols and does not apply heavy optimizations. This allows developers to easily map the running program back to the source code, making it easier to identify and fix logical errors, although the program runs slower.
    - *Release Build*: This configuration is used for final deployment to end-users. It removes debugging symbols and applies aggressive compiler optimizations to improve execution speed and reduce the memory footprint of the application.
2. **Breakpoints**: A breakpoint is a deliberate stopping point placed on a specific line of executable code. When the debugger runs and hits a breakpoint, the program

pauses execution. At this moment, developers can inspect the current values of variables, analyze the program's call stack, and evaluate expressions. This feature is essential for diagnosing complex runtime errors that are not visible through simple print statements.

3. **Refactoring**: Refactoring is the disciplined technique of restructuring existing code to improve its internal structure and readability without altering its external behavior. Common refactoring operations include renaming variables or functions, extracting code blocks into separate methods, and simplifying conditional logic. Regular refactoring ensures that the code remains clean, maintainable, and easier for multiple team members to understand over time.

4. **Source Code Collaboration (Git)**: Modern software development is a collaborative effort. Git is a distributed version control system that tracks changes in source files across multiple contributors. VS Code comes with built-in Git integration, which allows developers to initialize repositories, stage changes, commit snapshots, and synchronize their work with remote platforms like GitHub directly from the editor without switching to a command line.

5. **Command Palette**: The Command Palette is a unique, keyboard-driven feature of VS Code. Activated by pressing `Ctrl+Shift+P`, it provides quick access to almost every command available in the editor. Instead of navigating through complex menus, developers can simply type a command (e.g., "Format Document" or "Git: Push") and execute it instantly, drastically speeding up the workflow.

6. **IntelliSense**: IntelliSense is VS Code's intelligent code completion engine. It goes beyond simple text prediction by analyzing the code context, imported libraries, and variable types. As the developer types, it provides suggestions for variables, functions, methods, and parameters. It also shows documentation hints and function signatures, which helps reduce syntax errors, minimizes typos, and significantly accelerates the coding process.

OBJECTIVES

- To understand the purpose of an Integrated Development Environment (IDE).
- To explore the Build process in Debug and Release modes.
- To learn how breakpoints are used during debugging.
- To understand the concept of code refactoring.

- To explore Git integration for source code management.
- To use the Command Palette for accessing IDE commands.
- To understand the benefits of IntelliSense for code completion.

ALGORITHM

1. Launch Visual Studio Code on the system and open a new project folder.
2. Create a new source code file (e.g., `app.js`) and write a simple program containing basic functions and variables.
3. Observe the IntelliSense feature: start typing a known keyword (e.g., `con` for `console`) and note the automatic suggestions, parameter hints, and auto-completion that appear.
4. Open the Command Palette using `Ctrl+Shift+P` and search for a command such as "Format Document" to observe how quickly tasks can be executed without using the mouse.
5. Set a breakpoint by clicking on the left margin (gutter) next to a line of executable code.
6. Start the program in **Debug Build** mode using the "Run and Debug" panel. Allow the code to run until it hits the breakpoint and observe how the execution pauses.
7. While paused at the breakpoint, inspect the current values of local variables in the "Variables" pane and use the "Step Over" button to execute the code line by line.
8. Perform a **Refactoring** operation: right-click on a variable name used multiple times, select "Rename Symbol", and change its name to something more meaningful. Verify that the IDE updates all occurrences automatically.
9. Initialize a local Git repository by opening the Source Control panel and clicking "Initialize Repository". Stage the changes by clicking the "+" icon next to the file.
10. Write a commit message and commit the staged changes. Finally, connect to a remote GitHub repository and push the code to synchronize it (conceptually, or practically if credentials are set up).
11. Switch the configuration to **Release Build** (if applicable) and note the conceptual difference that the compiled output would be optimized for performance.
12. Observe and document the outputs and effectiveness of each explored feature.

CODE

JavaScript Example (Testing Breakpoints & IntelliSense)

```
function add(a, b) {
    return a + b;
}

function multiply(a, b) {
    return a * b;
}

let x = 10;
let y = 20;
let sum = add(x, y);
let product = multiply(x, y);

console.log("Sum:", sum);
console.log("Product:", product);
```

Git Commands (for Collaboration)

```
git init
git add .
git commit -m "Lab 2: IDE Features Exploration"
git remote add origin https://github.com/username/repository.git
git branch -M main
git push -u origin main
```

OUTPUT

The features of Visual Studio Code were successfully explored and implemented.

- **IntelliSense**: Provided automatic code suggestions and parameter hints while coding, reducing typing effort.
- **Breakpoints**: Successfully paused program execution at the specified line, allowing inspection of variables.
- **Refactoring**: The "Rename Symbol" feature changed all instances of a variable across the file instantly, improving code readability.

- **Git Collaboration**: The repository was initialized, changes were staged and committed, and the project was linked to a remote source.
- **Command Palette**: Executed various IDE commands quickly without using toolbar menus.
- **Build Modes**: Observed that Debug mode supports active troubleshooting, while Release mode is designed for performance optimization.

RESULT

The experiment was completed successfully. All the major features of Visual Studio Code, including Build configurations, Breakpoints, Refactoring, Git integration, Command Palette, and IntelliSense, were explored and used effectively. The understanding of how each feature contributes to a smooth, collaborative, and efficient development lifecycle was gained.

CONCLUSION

This lab provided practical, hands-on knowledge of the essential features of Visual Studio Code. It is evident that modern IDEs are more than just text editors; they are powerful ecosystems that streamline the entire software development process. The explored features—IntelliSense and Command Palette speed up daily coding tasks, Breakpoints and Debug/Release modes simplify error detection and performance tuning, Refactoring ensures code quality, and Git integration enables seamless teamwork. Mastering these tools is fundamental for any developer working in an Agile Software Development environment.